



Chapitre 2 : Implémentation des processus



1

1



Plan

- Complément en c
 - Concepts fondamentaux
 - Création des processus
 - Les tubes
 - Les signaux
- 

2

2

Complément en C

3

3

Complément en c

- Un premier programme en C

```
/* Un programme qui affiche bonjour. */
#include <stdio.h> //standard input/output library, ou stdio

int main()
{ printf("Bonjour.\n");
  return 0; //termine l'exécution du main et retourne un résultat de 0 }
```

- Variables

- Toute variable a un type, qui spécifie la taille et l'interprétation de la mémoire associée à la variable

```
#include <stdio.h>
int main()
{ int i; i = 1;
  printf("La variable i vaut %d.\n", i);
  i = 2;
  printf("La variable i vaut maintenant %d.\n", i);
  i = i + 1;
  printf("La variable i vaut enfin %d.\n", i);
  return 0;
}
```

4

4

Complément en c

- **Sortie formatée**
 - L'instruction *printf* permet de formater et d'écrire des données sur la sortie standard
 - `printf("Le carré de %d est %d.\n", i, i * i);`
- **Entrées**
 - L'instruction *scanf* permet de lire une valeur à partir de l'entrée standard

```
#include <stdio.h>
int main()
{
    int i;
    printf("Entrez un nombre: ");
    scanf("%d", &i);
    printf("Vous avez entré %d dont le carré est %d.\n", i, i * i);
    return 0;
}
```

5

5

Complément en c

- **Conditionnelles simples**
 - La conditionnelle *if* est une structure de contrôle. Elle consiste du mot clé *if* suivi d'une *condition* entre parenthèses et de deux branches séparées par le mot clé *else*. Si la condition est vraie, c'est la première branche qui est exécutée ; sinon, c'est la deuxième.

```
#include <stdio.h>
int main()
{
    int i;
    printf("Entrez un nombre: ");
    scanf("%d", &i);
    if(i <= 5) {
        printf("Inférieur ou égal à 5.\n");
    } else {
        printf("Pas inférieur ou égal à 5.\n");
    }
    return 0;
}
```

6

6

Complément en c

▪ Conditionnelles imbriquées

- Une conditionnelle peut aussi apparaître au sein d'une des deux branches d'une autre conditionnelle.

```
#include <stdio.h>
int main()
{
    int i;
    printf("Entrez un nombre: ");
    scanf("%d", &i);
    if(i <= 5) {
        printf("Inférieur ou égal à 5.\n");
    } else {
        printf("Pas inférieur ou égal à 5.\n");
    }
    return 0;
}
```

7

7

Complément en c

▪ Boucles définies

- Une boucle est une structure de contrôle qui sert à exécuter le même bloc de code de multiples fois
- Une boucle définie, ou boucle for, est une boucle dont l'exécution est contrôlée par un compteur de boucle dont la valeur varie entre deux valeurs connues avant d'entrer dans la boucle
- **for(e1 ; e2 ; e3) bloc**
 - l'expression *e1*, dite initialisation, est exécutée ;
 - le test *e2* est évalué; s'il est faux, on sort de la boucle, et l'exécution de la boucle est terminée ;
 - le corps de la boucle est exécuté ;
 - l'expression *e3* est exécutée ;
 - on recommence à l'étape 2.

```
int main()
{ int i;
  for(i = 1; i <= 10; i = i + 1)
  { printf("J'ai collé %d timbres.\n", i);
    printf("Il m'en reste %d.\n", 10 - i);
  }
}
```

8

8

Complément en c

▪ Quelques abréviations

- l'expression `i++` est équivalente à `i = i + 1 ;`
- l'expression `i--` est équivalente à `i = i - 1 ;`
- l'expression `i += c` est équivalente à `i = i + c ;`
- l'expression `i -= c` est équivalente à `i = i - c ;`

❑ Terminaison prématurée à l'aide de `return`

- Il est parfois nécessaire d'interrompre l'exécution d'une boucle de façon prématurée,

```
#include<stdio.h>
int main()
{ int i; double x, somme;
  somme = 0.0;
  for(i = 0; i < 4; i = i + 1) {
    scanf("%lf", &x);
    if(x > -1.0E10 && x < 1.0E-10) {
      printf("Division par zéro !\n"); return 1; //on utilise aussi Break
    }
    somme = somme + 1.0 / x;
  }
  printf("La somme des inverses est %lf.\n", somme);
  return 0;
}
```

9

9

Complément en c

▪ Les directives

- Figurent sur les lignes commençant par `#`
- Les principales directives : **`#include`** qui permet d'inclure un fichier extérieur
 - **`#include <nom_de_fichier>`** le fichier est recherché dans un répertoire standard (`/usr/include`)
 - **`#include "nom_de_fichier"`** le fichier est recherché dans le répertoire de travail puis, s'il n'y est pas, dans le répertoire standard
- Les principales directives : **`#define`** permet de définir une macro
 - Macro simple : **`#define nom_macro [texte_de_replacement]`**, ex, `#define TAILLE 500`
 - Macro-fonction : **`#define nom(param_1, ..., param_n) texte`**, ex, `#define min(a,b) (((a) < (b)) ? (a) : (b))`

10

10

Complément en c

❑ Les tableaux à une ou plusieurs dimensions

- Définition : *type nom[expr_constante];*
- Définition avec initialisation : *type nom[expr] = {val0, val1, ..., valn};*
- Déclaration sans définition : *type nom[];*
- Les index d'un tableau de *n* éléments varient entre 0 et n-1

```
#include <stdio.h>
#define MAXTAB 5
int min(int tab[],int nb)
{
    int i, min=tab[0];
    for (i=1; i<nb; i++)
        if (tab[i] < min) min = tab[i];
    return min;
}
int main()
{
    int tab[MAXTAB] = { 106, 20, 34, 4, 15};
    printf("Le minimum est : %d\n", min(tab,MAXTAB));
}
```

11

11

Complément en c

❑ Les structures

- L'accès aux champs d'une structure s'effectue directement par l'opérateur « . » : **var1.a1** pour accéder au champ **a1** de la variable **var1**

```
#define MAXRUE 80
#define MAXVILLE 25
struct adresse {
    int num;
    char rue[MAXRUE];
    char ville[MAXVILLE];
    unsigned long code;
};
...
struct adresse uneAdresse;
...
uneAdresse.num = 50;
uneAdresse.code = 54000;
...
```

12

12

Complément en c

❑ Les chaînes de caractères

- La bibliothèque standard contient un assortiment très complet de fonctions permettant de manipuler les chaînes de caractères comme des tableaux de caractères
- il faut inclure **string.h** (parfois **stdlib.h**)
- ces fonctions utilisent le type **size_t** synonyme de **unsigned**
- dans les prototypes de fonction, **const char *** désignent les arguments de type chaîne non modifiés par la fonction et **char *** ceux modifiés
- ces fonctions reconnaissent la fin d'une chaîne dès la rencontre du premier caractère **\0**
- **Principales fonctions**
 - **strlen(str)** : Calcule la longueur d'une chaîne
 - **strcpy(dest, src)** : Copie la chaîne src vers dest
 - **strcat(dest, src)** : Concatène la chaîne src à la fin de dest
 - **strcmp(s1, s2)** : Compare deux chaînes de caractères

```
/* affecter une chaîne */
strcpy(uneAdresse.rue, "rue de la Commanderie");
...
/* comparer l'égalité entre deux chaînes */
if (!strcmp(uneAdresse.ville, "Nancy"))
...
```

13

13

Complément en c

❑ Pointeurs

- un pointeur est un couple (adresse en mémoire, type de donnée)
- exemple : un pointeur sur un entier est donc l'adresse d'un entier dans la mémoire
- déclaration d'une variable de type pointeur : **type_pointé *ptr** ; où **ptr** est une variable qui contient l'adresse en mémoire d'un objet du **type_pointé**
- on accède à la valeur de l'objet pointé par **ptr** en utilisant l'opérateur d'indirection ***(ptr)**
- l'opérateur de prise d'adresse **&** permet d'accéder à l'adresse d'un objet

```
/* attention, variable de type pointeur
sur une structure adresse */
struct adresse *monAdresse;
...
/* elle pointe sur un objet alloué */
monAdresse = &uneAdresse;

/* je peux modifier la valeur d'un champ */
(*monAdresse).num = 35;

/* c'est la même chose que */
monAdresse->num = 35;
...
```

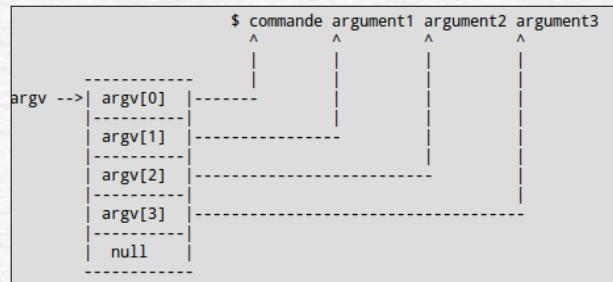
14

14

Complément en c

❑ Le principe des arguments de la ligne de commande

- Un programme C récupère les informations tapées sur la ligne commande du Shell par l'intermédiaire des variables prédéfinies `argc` et `argv`.
- Ces variables doivent être déclarées en argument de la fonction `main()` qui sera le point d'entrée initial du programme C



15

15

Complément en c

❑ Les arguments de la ligne de commande: syntaxe

- On peut écrire de nouvelles commandes qui peuvent être appelées avec des paramètres
- La fonction `main()` s'écrit de la manière suivante :

```
int main(int argc, char **argv)
{
    ...
}
```

- la valeur de **argc** est le nombre de chaîne de caractères de la ligne de commande
- **argv** est un tableau de pointeurs sur des caractères pointant sur la première chaîne de caractères de la ligne de commande : **argv[0]** pointe sur le nom du programme, **argv[1]** sur le premier paramètre, ...
- Exemple: si un fichier `monprog.c` permet de générer un exécutable `monprog` à la compilation (`cc monprog.c -o monprog`) on peut invoquer le programme `monprog` avec des arguments : `./monprog arg1 arg2 arg3`
- La commande **cp** du bash prend deux arguments

16

16

Complément en c

❑ Les arguments de la ligne de commande

- la valeur de **argc** est le nombre de chaîne de caractères de la ligne de commande
- **argv** est un tableau de pointeurs sur des caractères pointant sur la première chaîne de caractères de la ligne de commande : **argv[0]** pointe sur le nom du programme, **argv[1]** sur le premier paramètre, ...

```
#include <stdio.h>

int main(int argc char *argv[])
{
    int i;
    if (argc == 1)
        puts("Le programme n'a reçu aucun argument");
    if (argc >= 2)
    {
        puts("Le programme a reçu les arguments suivants :");
        for (i=1 ; i<argc ; i++)
            printf("Argument %d = %s\n", i, argv[i]);
    }
    return 0;
}
```

17

17

Concepts fondamentaux

18

18

Concepts fondamentaux des processus

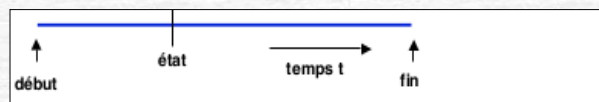
- Processus Linux/Unix
 - Un processus est une instance d'un programme en train de s'exécuter, une tâche.
 - Il possède un numéro unique sur le système pid
 - Chaque processus appartient à un utilisateur et un groupe et aux droits qui leur sont associés
 - Sous shell, un processus est créé pour exécuter chacune des commandes
 - Le shell est le processus père de toutes les commandes.
- Statut d'un processus
 - **Running**: le processus s'exécute
 - **Waiting**: Attend quelque chose pour s'exécuter
 - **Ready**: le processus a tout pour s'exécuter sauf le processeur
 - **Suspendu**: Arrêté
 - **Zombie**: état particulier (L'exécution est terminée, mais le parent ne l'a pas encore confirmée)

19

19

Concepts fondamentaux des processus

- Le multitâche sous Unix/Linux
 - Unix est un système multitâche: il peut exécuter plusieurs progs à la fois
 - Le shell crée un nouveau processus pour exécuter chaque commande
 - Différence entre processus et programme :
 - le programme est une description statique
 - le processus est une activité dynamique (il a un début, un déroulement et une fin, il a un état qui évolue au cours du temps)
 - Par exemple, l'exécution d'un programme et la copie d'un fichier sur disque



- Exemple
 - **xclock**: lancement d'un processus en 1er plan (foreground)
 - **ctrl+c**: l'arrêt définitif d'un processus
 - **xclock&**: lancement d'un processus en 2ieme plan (background)
 - **^z**: un signal qui suspend l'exécution sans détruire le processus correspondant
 - **bg**: pour afficher les processus en BG
 - **fg**: pour afficher les processus en FG

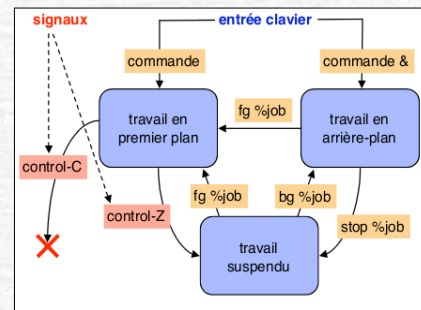
20

20

Concepts fondamentaux des processus

- Le programme tourne au premier plan :
 - ^Z le suspend ;
 - ^C l'interrompt.
- Le programme est suspendu :
 - fg le passe au premier plan ;
 - bg le passe en arrière-plan.
- ❖ Le programme tourne en arrière-plan :
 - si c'est le seul dans ce cas, fg le passe au premier plan ;
 - sinon, c'est plus compliqué.
- Le programme ne tourne pas :
 - il n'y a rien à faire...

- Travail (job) = processus lancé par une commande au shell
- Seul le travail en premier plan peut recevoir des signaux du clavier.
- Les autres sont manipulés par des commandes



21

21

Concepts fondamentaux des processus

- Exemple

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

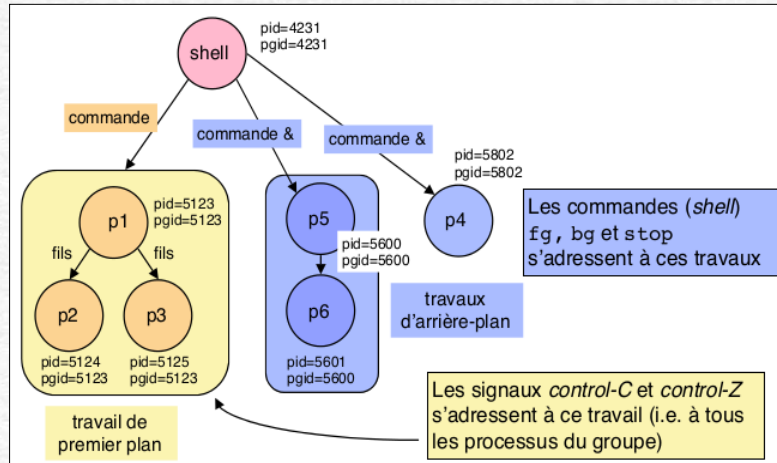
int main() {
    printf("processus %d, groupe %d\n", getpid(), getppid());
    while(1);
}
```

```
saad@saad:~/Desktop/System Prog/Chap2$ gcc prog4.c -o prog4
saad@saad:~/Desktop/System Prog/Chap2$ ./prog4 & ./prog4 & ps
[1] 9092
[2] 9093
Processus 9093, groupe 9093
Processus 9092, groupe 9092
  PID TTY          TIME CMD
 5129 pts/0        00:00:00 bash
  9092 pts/0        00:00:00 prog4
  9093 pts/0        00:00:00 prog4
  9094 pts/0        00:00:00 ps
saad@saad:~/Desktop/System Prog/Chap2$ fg %1
./prog4
^Z
[1]+  Stopped                  ./prog4
saad@saad:~/Desktop/System Prog/Chap2$ jobs
[1]+  Stopped                  ./prog4
[2]-  Running                  ./prog4 &
saad@saad:~/Desktop/System Prog/Chap2$ kill %1
[1]+  Stopped                  ./prog4
saad@saad:~/Desktop/System Prog/Chap2$ jobs
[1]+  Terminated              ./prog4
[2]-  Running                  ./prog4 &
saad@saad:~/Desktop/System Prog/Chap2$ kill 9093
saad@saad:~/Desktop/System Prog/Chap2$ jobs
[2]+  Terminated              ./prog4
saad@saad:~/Desktop/System Prog/Chap2$
```

22

22

Concepts fondamentaux des processus



23

23

Concepts fondamentaux des processus

- La commande **ps**:
 - PID (process identifier) : c'est le numéro du processus.
 - TT : indique le terminal dans lequel a été lancé le processus. Un point d'interrogation signifie que le processus n'est attaché à aucun terminal (par exemple les démons).
 - STAT : indique l'état du processus :
 - R : actif (running)
 - S : non activé depuis moins de 20 secondes (sleeping)
 - I : non activé depuis plus de 20 secondes (idle)
 - T : arrêté (suspendu)
 - Z : zombie
 - TIME : indique le temps machine utilisé par le programme (et non pas le temps depuis lequel le processus a été lancé !).

24

24

Concepts fondamentaux des processus

- Les options de la commande **ps** :
 - **a (all)** : donne la liste de tous les processus, y compris ceux dont vous n'êtes pas propriétaire.
 - **g (global, général...)** : donne la liste de tous les processus dont vous êtes propriétaire.
 - **u (user, utilisateur)** : donne davantage d'informations (nom du propriétaire, heure de lancement, pourcentage de mémoire occupée par le processus, etc.).
 - **x** : affiche aussi les processus qui ne sont pas associés à un terminal.
 - **agux** : est en fait souvent utilisé pour avoir des informations sur tous les processus.
- Commande **top**
 - Affiche les mêmes informations, mais de façon dynamique : elle indique en fait par ordre décroissant le temps machine des processus, les plus gourmands en premier.

25

25

Concepts fondamentaux des processus

- Tuer les processus
 - **^c; ^z**
 - **Kill pid**: tuer un processus
 - **Kill -9 pid**: imposer l'arrêt immédiat du processus
- Vie et mort des processus:
 - **Début** : création par un autre processus - par `fork()`
 - **Fin**: auto-destruction (à la fin du programme) - par `exit()`; destruction par un autre processus - par `kill()`; certains processus ne se terminent pas ("démons", réalisant des fonctions du système)
- Dans Unix/Linux
 - **Dans le langage de commande**: un processus est créé pour l'exécution de chaque commande. On peut créer des processus pour exécuter des commandes en (pseudo)-parallèle :
 - `prog1 & prog2 & /*` crée deux processus pour exécuter `prog1` et `prog2` */
 - `prog1 & prog1 & /*` crée deux exécutions parallèles de `prog1` */
 - **Au niveau des appels système**: un processus est créé par une instruction `fork()` (voir plus loin)

26

26

Concepts fondamentaux des processus

- Autres commandes
 - **ps** : liste des processus et de leurs caractéristiques
 - **htop** : liste dynamique des processus et de ce qu'ils consomment
 - **pgrep** : récupération d'une liste de processus par expression régulière
 - **pidof** : récupération du pid d'un processus recherché
 - **fuser** : informations sur les file descriptor d'un processus
 - **lsof** : idem
 - **pmap** : afficher le mapping mémoire d'un processus
 - **strace** : liste les appels système du processus
 - **ltrace** : liste les appels de fonction de bibliothèques dynamiques du processus
 - **pstack** : affiche la pile d'appel du processus
 - **gdb** : pour tout savoir et même modifier l'action d'un processus.
 - **kill** : envoyer un signal à un processus connaissant son pid
 - **killall** : envoie un signal à tous les processus portant un certain nom
 - **pkill** : envoie un signal aux processus matchant une expression régulière
 - **ctrl-z** : envoie le signal STOP au processus en avant plan du shell en cours
 - **fg, bg** : envoie le signal CONT à un processus stoppé du shell en cours

27

27

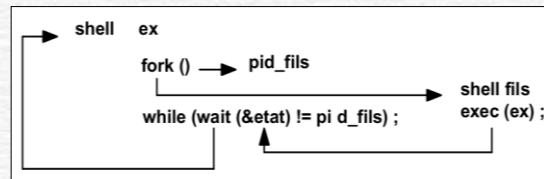
Création des processus

28

28

Création de processus avec le Shell

- Soit **ex** un fichier de commandes exécutable.
- 1er cas : on tape \$ **ex**, voici ce qui se passe :
 1. le shell lit la commande **ex**
 2. il se duplique au moyen de la fonction **fork** ; il existe alors un shell père et un shell fils
 3. grâce à la fonction **exec**, le shell fils recouvre son segment de texte par celui de **ex** qui s'exécute.
 4. le shell père récupère le pid du fils retourné par **fork ()**
 5. à la fin de l'exécution de **ex**, le shell père reprend son exécution.



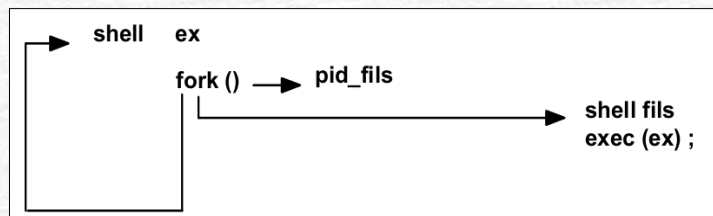
- La fonction **wait** lui permet de connaître son achèvement : **wait (&etat)** retourne le **pid** du processus fils à son achèvement (**etat** : entier).

29

29

Création de processus avec le Shell

- 2ème cas : on tape **ex&**, mais n'accède pas à l'entrée standard réservée au shell), voici ce qui se passe :
 - 1. à 4. comme ci-dessus
 - 5. le shell père reçoit le **pid** du fils, émet un prompt, puis reprend son activité sans attendre la fin du fils



- un processus lancé en arrière-plan ne réagit pas aux interruptions émises au clavier

30

30

La norme Posix

- La norme Posix (Portable Operating System Interface) définit un nombre relativement petit d'appels système pour la gestion de processus :
 - **pid_t fork()** : Création de processus fils.
 - **int execl(), int execlp(), int execvp(), int execl(), int execv()** : Les services **exec()** permettent à un processus d'exécuter un programme (code) différent.
 - **pid_t wait()** : Attendre la terminaison d'un processus
 - **void exit()** : Finir l'exécution d'un processus.
 - **pid_t getpid()** : Retourne l'identifiant du processus.
 - **pid_t getppid()** : Retourne l'identifiant du processus père.
 - En Linux, le type **pid_t** correspond normalement à un **long int**.

31

31

Création de processus avec « system() »

- Il y a une façon de créer un sous-processus en Unix/Linux, en utilisant la commande **system()**, de la bibliothèque standard de C `<stdlib.h>`.
 - Comme arguments elle reçoit le nom de la commande (et peut-être une liste d'arguments) entre guillemets.
 - Dans un shell. Il faut retenir que **system()** n'est pas un appel système, mais une fonction C.
 - Ce qui rend l'utilisation de la fonction **system()** moins performante qu'un appel système de création de processus !!!

```
#include <stdlib.h>
```

```
int main()
```

```
{ int return_value ;
```

```
/* retourne 127 si le shell ne peut pas s'exécuter, retourne -1 en cas d'erreur, autrement retourne le code de la commande */
```

```
return_value = system("ls -l") ; return return_value ;
```

```
}
```

```
saad@saad:~/Desktop/System Prog/Chap2$ ./prog5
total 120
-rwxrwxr-x 1 saad saad 15960 Feb 13 11:04 prog1
-rw-rw-r-- 1 saad saad 68 Feb 13 11:04 prog1.c
-rwxrwxr-x 1 saad saad 16064 Feb 13 11:58 prog2
-rw-rw-r-- 1 saad saad 170 Feb 13 11:58 prog2.c
-rwxrwxr-x 1 saad saad 16000 Feb 13 12:43 prog3
-rw-rw-r-- 1 saad saad 285 Feb 13 12:43 prog3.c
-rwxrwxr-x 1 saad saad 16048 Feb 13 13:04 prog4
-rw-rw-r-- 1 saad saad 144 Feb 13 13:04 prog4.c
-rwxrwxr-x 1 saad saad 15960 Feb 15 11:51 prog5
-rw-rw-r-- 1 saad saad 110 Feb 15 11:51 prog5.c
-rwxrwxr-x 1 saad saad 16048 Feb 15 22:36 whoami
-rw-rw-r-- 1 saad saad 131 Feb 15 22:05 whoami.c
```

32

32

Création de processus avec « system() »

- On peut lancer un programme à l'intérieur d'un autre programme et ainsi créer un nouveau processus en utilisant la bibliothèque system.

```
#include <stdlib.h>
```

```
int system (const char *string)
```

- Exemple: un programme qui exécute **ps**.

```
#include <stdio.h>
#include <stdlib.h>

int main ()
{
    printf ( "executer ps avec system\n" );
    system ("ps ax");
    printf ("c'est fait\n");
    exit(0);
}
```

33

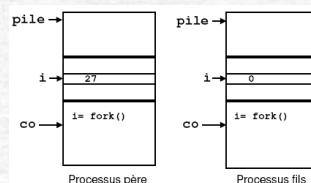
33

Création de processus avec « fork() »

- Création d'un processus
 - fork()** crée un processus fils, une copie **exacte** du processus original
 - Ce processus possède sa propre image mémoire (modifier une variable dans une des processus ne la modifie pas dans l'autre)

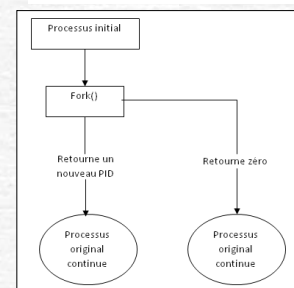
```
#include <sys/types.h>
#include <unistd.h>
pid_t fork(void);
```

```
switch (n=fork()) {
case -1: perror("fork"); _
case 0: /* code du fils */
default: /* code du père */
}
```



Problème !!! les deux processus père et fils exécutent le même code. Comment distinguer alors le processus père du processus ls ?

Solution: on regarde la valeur de retour de **fork()**

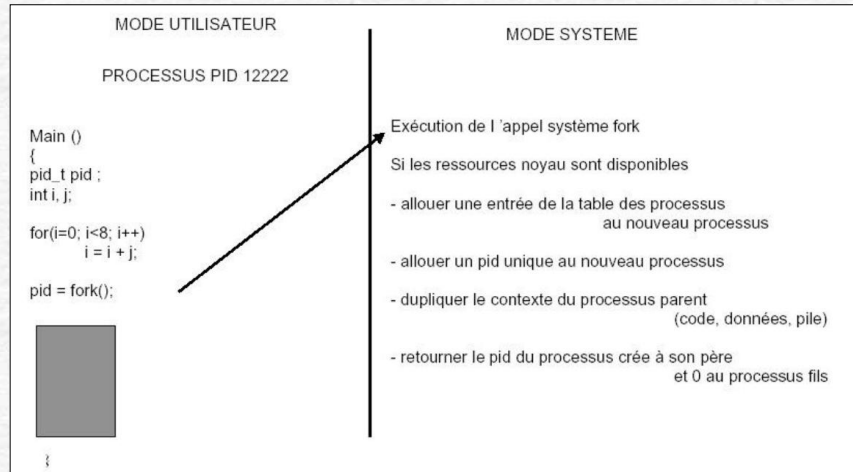


34

34

Création de processus avec « fork() »

■ La primitive Fork()



35

35

Création de processus avec « fork() »

■ Création d'un processus

- Les deux processus père et fils exécutent le même code. Comment distinguer alors le processus père du processus fils?
- Pour résoudre ce problème, on regarde la valeur de retour de fork(), qui peut être :
 - 0 pour le processus fils
 - Strictement positive pour le processus père et qui correspond au pid du processus fils
 - Négative si la création de processus a échoué, s'il n'y a pas suffisamment d'espace mémoire ou si bien le nombre maximal de créations autorisées est atteint.

```
#include <sys/types.h>
#include <unistd.h>
pid_t getpid(void);
pid_t getppid(void);
```

■ Identifiant de processus

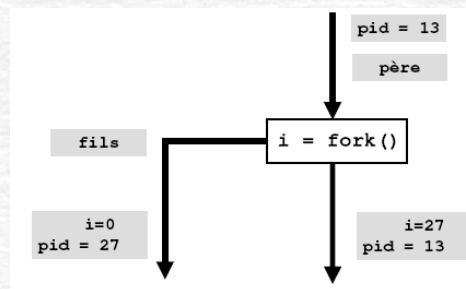
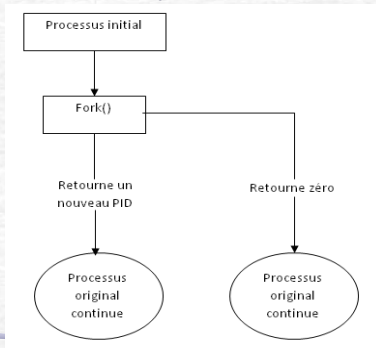
- Chaque processus possède un identifiant, son **pid**
- La fonction **getpid** retourne le pid du processus actif
- La fonction **getppid** retourne le pid du processus père

36

36

Création de processus avec « fork() »

- Création d'un processus
 - Après l'appel système **fork()**, la valeur de pid reçoit la valeur 0 dans le processus fils mais elle est égale à l'identifiant du processus fils dans le processus père
 - Le processus créé (fils) est un clone (copie conforme) du processus créateur (père). Le père et le fils ne se distinguent que par le résultat rendu par fork()
 - pour le père : le numéro du fils (ou -1 si création impossible), pour le fils : 0



37

37

Création de processus avec « fork() »

```
main ()
{
    int pid_fils;
    bloc1
    if ((pid_fils = fork()) == 0)
        bloc2
    else
        bloc3
}
```

- Le processus père exécutera le **bloc 1**.
- Puis il créera un fils identique par **fork** et exécutera le **bloc3** puisque **fork** lui retournera une valeur non nulle.
- Le processus fils n'exécutera pas le **bloc1** car à sa création, son compteur ordinal pointerait sur la ligne contenant **fork**. Comme **fork** lui retourne 0, il exécutera le seul **bloc2**.

38

38

Création de processus avec « fork() »

Exemple

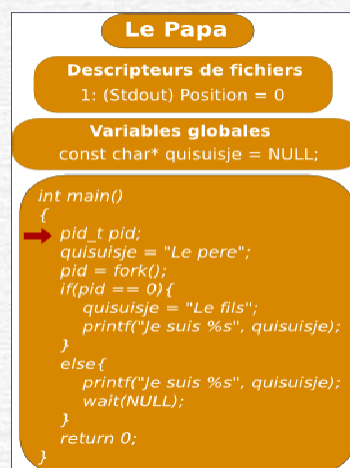
```
#include<stdio.h>
#include<sys/times.h>
Int main()
{
    int pid;
    Char quisuisje="le pere";
    Pid=fork();
    If (pid==0)
    { Quisuisje="le fils";
      Printf("je suis le %s ", quisuisje );
    }
    else
    {
      Printf("je suis le %s ", quisuisje );
      Wait(NULL);
    }
    Return 0;
}
```

39

39

Création de processus avec « fork() »

Création de processus

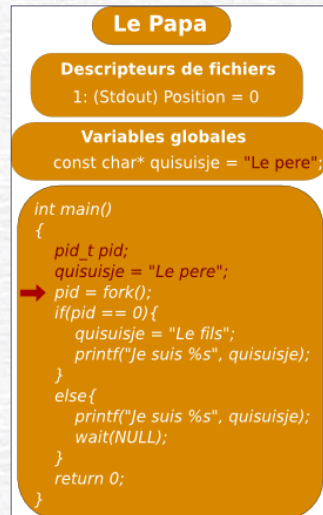


40

40

Création de processus avec « fork() »

■ Création de processus

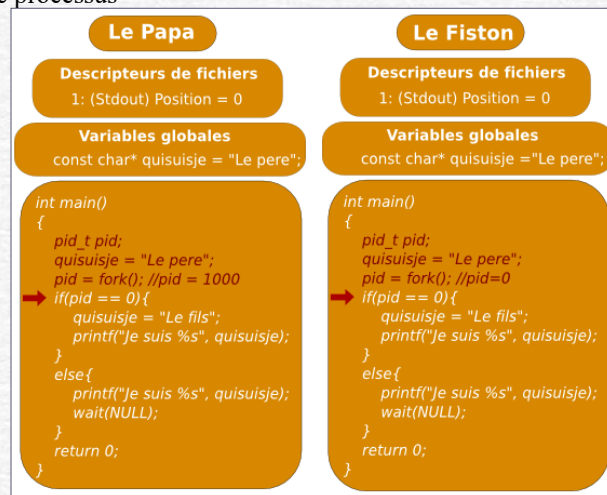


41

41

Création de processus avec « fork() »

■ Création de processus



42

42

Création de processus avec « fork() »

■ Création de processus

Le Papa	Le Fiston
Descripteurs de fichiers 1: (Stdout) Position = 0	Descripteurs de fichiers 1: (Stdout) Position = 0
Variables globales const char* quisuisje = "Le pere";	Variables globales const char* quisuisje = "Le pere";
<pre>int main() { pid_t pid; quisuisje = "Le pere"; pid = fork(); //pid = 1000 if(pid == 0) { quisuisje = "Le fils"; printf("Je suis %s", quisuisje); } else { printf("Je suis %s", quisuisje); wait(NULL); } return 0; }</pre>	<pre>int main() { pid_t pid; quisuisje = "Le pere"; pid = fork(); //pid=0 if(pid == 0) { quisuisje = "Le fils"; printf("Je suis %s", quisuisje); } else { printf("Je suis %s", quisuisje); wait(NULL); } return 0; }</pre>

43

43

Création de processus avec « fork() »

■ Création de processus

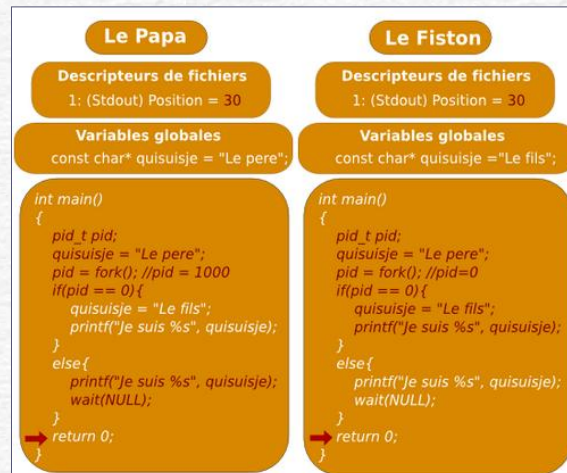
Le Papa	Le Fiston
Descripteurs de fichiers 1: (Stdout) Position = 15	Descripteurs de fichiers 1: (Stdout) Position = 15
Variables globales const char* quisuisje = "Le pere";	Variables globales const char* quisuisje = "Le fils";
<pre>int main() { pid_t pid; quisuisje = "Le pere"; pid = fork(); //pid = 1000 if(pid == 0) { quisuisje = "Le fils"; printf("Je suis %s", quisuisje); } else { printf("Je suis %s", quisuisje); wait(NULL); } return 0; }</pre>	<pre>int main() { pid_t pid; quisuisje = "Le pere"; pid = fork(); //pid=0 if(pid == 0) { quisuisje = "Le fils"; printf("Je suis %s", quisuisje); } else { printf("Je suis %s", quisuisje); wait(NULL); } return 0; }</pre>

44

44

Création de processus avec « fork() »

■ Création de processus



45

45

Création de processus avec « fork() »

■ Création d'un processus

- Après l'appel système **fork()**, la valeur de pid reçoit la valeur 0 dans le processus fils mais elle est égale à l'identifiant du processus fils dans le processus père
- Le processus créé (fils) est un clone (copie conforme) du processus créateur (père). Le père et le fils ne se distinguent que par le résultat rendu par fork()
 - Le code de retour de fork vaut 0 dans le fils; Dans le père, c'est l'identifiant du processus (pid) fils; En cas d'erreur, ce code de retour vaut -1

1 programme, 2 processus

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
int main ()
{ if (fork() != 0) { printf("je suis le père, mon PID est %d\n", getpid());
  } else {
    printf("je suis le fils, mon PID est %d\n", getpid()); /* en général exec (exécution d'un
nouveau programme) */
  }
}
```

46

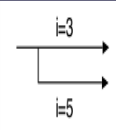
46

Création de processus avec « fork() »

- Création d'un processus: 1 programme, 2 processus, donc 2 mémoires virtuelles, 2 jeux de données

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
int main ()
{ int i ;
  if (fork() != 0) {
    printf("je suis le père, mon PID est %d\n", getpid());
    i = 3;
  } else {
    printf("je suis le fils, mon PID est %d\n", getpid());
    i = 5;
  }
  printf("pour %d, i = %d\n", getpid(), i);
}
```

```
saad@NSAAD:~/Desktop/Chap2$ ./perefiles
je suis le père, mon PID est 10935
pour 10935, i=3
je suis le fils, mon PID est 10936
pour 10936, i=5
```



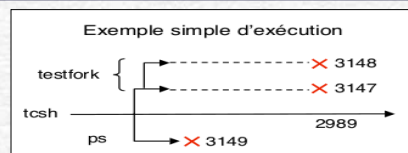
47

47

Création de processus avec « fork() »

- Création d'un processus

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
int main ()
{ int i ;
  if (fork() != 0) {
    printf("je suis le père, mon PID est %d\n", getpid());
    sleep(10) /* blocage pendant 10 seconds*/; exit(0);
  } else {
    printf("je suis le fils, mon PID est %d\n", getpid());
    sleep(10) /* blocage pendant 10 seconds*/; exit(0);
  }
}
```



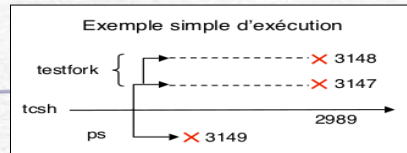
48

48

Création de processus avec « fork() »

■ Création d'un processus

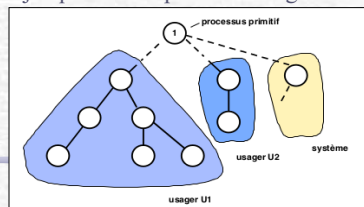
```
saad@NSAAD:~/Desktop/Chap2$ ./sleeping & ps
[3] 11003
je suis le père, mon PID est 11003
je suis le fils, mon PID est 11005
  PID TTY          TIME CMD
 2206 pts/2        00:00:00 bash
 11003 pts/2        00:00:00 sleeping
 11004 pts/2        00:00:00 ps
 11005 pts/2        00:00:00 sleeping
[2] Done
saad@NSAAD:~/Desktop/Chap2$ ./sleeping
saad@NSAAD:~/Desktop/Chap2$ ./sleeping & ps
[4] 11006
je suis le père, mon PID est 11006
je suis le fils, mon PID est 11008
  PID TTY          TIME CMD
 2206 pts/2        00:00:00 bash
 11003 pts/2        00:00:00 sleeping
 11005 pts/2        00:00:00 sleeping
 11006 pts/2        00:00:00 sleeping
 11007 pts/2        00:00:00 ps
 11008 pts/2        00:00:00 sleeping
```



49

Hiérarchie de processus et interactions entre processus

- Synchronisation entre un processus père et ses fils
 - Le fils termine son exécution par **exit(statut)**, où **statut** est un code de fin (par convention : 0 si normal, sinon code indiquant une erreur)
 - Un processus père peut attendre la fin de l'exécution d'un fils par la primitive : **pid_t wait(int *ptrStatut)**. La variable (facultative) **ptrStatut** recueille le statut, **wait** renvoie le PID du fils.
 - On peut aussi utiliser **pid_t waitpid(pid_t pid, int *ptrStatut)** pour attendre la fin de l'exécution d'un fils spécifié pid
- Envoyer un signal à un autre processus
 - Sera vu plus tard en détail. Pour le moment, on peut utiliser **kill pid** au niveau du langage de commande, pour tuer un processus spécifié pid
- Faire attendre un processus
 - **sleep(n)** : se bloquer pendant n secondes
 - **pause**: se bloquer jusqu'à la réception d'un signal envoyé par un autre processus

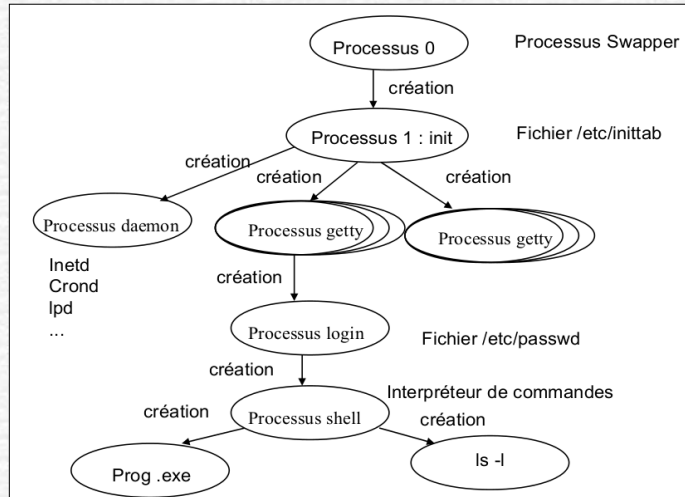


50

50

Hiérarchie de processus et interactions entre processus

- Le système Linux repose sur ce concept arborescent

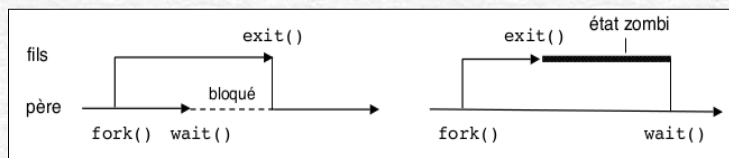


51

51

Synchronisation entre père et fils

- Quand un processus se termine, il délivre un code de retour (paramètre de la primitive `exit()`). Par exemple `exit(1)` renvoie le code de retour 1.
- Un processus père peut attendre la fin d'un ou plusieurs fils en utilisant `wait()` ou `waitpid()`. Tant que son père n'a pas pris connaissance de sa terminaison par l'une de ces primitives, un processus terminé reste dans un état dit **zombi**.
 - Un processus zombi ne peut plus s'exécuter, mais consomme encore des ressources. Il faut éviter de conserver des processus dans cet état.



52

52

Synchronisation entre père et fils

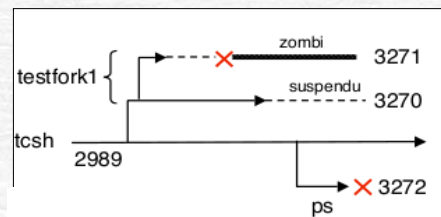
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main ()
{
    if (fork() != 0) {
        printf("je suis le père, mon PID est %d\n", getpid());
        while (1) ; /* boucle sans fin sans attendre le fils */
    } else {
        printf("je suis le fils, mon PID est %d\n", getpid());
        sleep(2) /* blocage pendant 10 seconds*/;
        printf("fin du fils\n");
        exit(0);
    }
}
```

53

53

Synchronisation entre père et fils



Zombie

```
saad@NSAAD:~/Desktop/Chap2$ ./synchro
je suis le père, mon PID est 11208
je suis le fils, mon PID est 11209
fin du fils
^Z
[1]+  Stopped                  ./synchro
saad@NSAAD:~/Desktop/Chap2$ ps aux | grep synchro
saad  11208 21.4  0.0   2680  1536 pts/2    T   20:21
0:03  ./synchro
saad  11209  0.0  0.0      0     0 pts/2    Z   20:21
0:00  [synchro] <defunct>
saad  11211  0.0  0.0   4088  2048 pts/2    S+  20:21
0:00  grep --color=auto synchro
saad@NSAAD:~/Desktop/Chap2$
```

54

54

Synchronisation entre père et fils

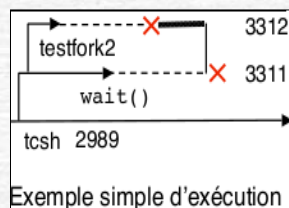
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main ()
{ pid_t fils; int statut;
  if (fork() != 0) {
    printf("je suis le père, mon PID est %d\n", getpid());
    fils = wait(&statut);
    if (WIFEXITED(statut)) {
      printf("%d : mon fils %d s'est terminé avec le code %d\n", getpid(), fils,
WEXITSTATUS(statut)); }
    exit(0);
  } else {
    printf("je suis le fils, mon PID est %d\n", getpid());
    sleep(10) /* blocage pendant 10 seconds */;
    printf("fin du fils\n");
    exit(1);
  }
}
```

55

55

Synchronisation entre père et fils

```
saad@NSAAD:~/Desktop/Chap2$ ./synchro2
je suis le père, mon PID est 11263
je suis le fils, mon PID est 11264
fin du fils
11263 : mon fils 11264 s'est terminé avec le code 1
saad@NSAAD:~/Desktop/Chap2$ ps
  PID TTY          TIME CMD
  2206 pts/2    00:00:00 bash
  11265 pts/2    00:00:00 ps
```



56

56

Exécution d'un programme spécifié

- La primitive `exec` :
 - sert à faire exécuter un nouveau programme par un processus
 - elle est souvent utilisée immédiatement après la création d'un processus
 - son effet est de "recouvrir" la mémoire virtuelle du processus par le nouveau programme et de lancer celui-ci en lui passant des paramètres spécifiés dans la commande.
- Diverses variantes d'`exec` existent selon le mode de passage des paramètres (tableau, liste, passage de variables d'environnement).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main ()
{
    if (fork() == 0) { execl("bin/ls", "ls", "-a", NULL); }
    else { wait(NULL); }
    exit(0);
}
```

le fils exécute :

```
/bin/ls -a ..
```

le père attend la fin
du fils

57

57

Exemple de la fonction `fork()`

- Exemple 1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{ pid_t v;
  v=fork();
  if(v==0) printf("Je suis le fils avec pid %d\n", getpid() );
  else if (v > 0)
      printf ("Je suis le pere avec pid %d\n", getpid() );
  else
      printf ("Erreur dans la creation du fils \n" );
}
```

```
saad@NSAAD:~/Desktop/Chap2$ gcc exemple1.c -o exemple1
saad@NSAAD:~/Desktop/Chap2$ ./exemple1
Je suis le pere avec pid 11325
Je suis le fils avec pid 11326
```

58

58

Exemple de la fonction fork()

Exemple 2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main ( void )
{
    int i , j , k , n=5;
    pid_t fils_pid ;
    for (i=1;i<n;i++)
    {
        fils_pid=fork();
        if (fils_pid>0) // c'est le pere
            break ;
        printf ( " Processus %d avec pere %d\n",getpid (), getppid () ) ;
    }
}
```

59

59

Exemple de la fonction fork()

Exemple 2

- Après l'exécution de « exemple2.c » on obtient le résultat suivant

```
saad@NSAAD:~/Desktop/Chap2$ ./exemple2
Processus 11423 avec pere 11422
Processus 11424 avec pere 11423
saad@NSAAD:~/Desktop/Chap2$ Processus 11425 avec pere 2203
Processus 11426 avec pere 11425
```

- Ce programme créera une chaîne de (n-1) processus
- Le problème ici est que dans quelques cas le processus père se termine avant le processus fils
- Comme tout processus doit avoir un parent, le processus orphelin est donc "adopté" par le processus dont le pid est 2203 (le processus init).
- Ainsi, on peut remarquer qu'au moment où les processus affichent le numéro de leur père, ils ont déjà été adoptés par le processus init

60

60

Exemple de la fonction fork()

- Exemple 3: le programme exemple3.cc suivant créera deux processus qui vont modifier la variable a

```
#include <sys/types.h> //type pid_t
#include <unistd.h> // pour fork
#include <stdio.h> // pour perror, printf
int main (void)
{ pid_t p ; int a = 20 ;
  // Création d'un fils
  switch ( p = fork ( ) )
  { case -1 : // le fork a echoue
    perror ( " le fork a echoue ! " ) ; break ;
    case 0 : // Il s'agit du processus fils
      printf ("ici processus fils, le PID %d. \n" , getpid ( ) ) ;
      a += 10 ; break ;
    default : // Il s'agit du processus pere
      printf ("ici processus pere, le PID %d.\n" , getpid ( ) ) ;
      a += 100 ;
  } // les deux processus exécutent cette instruction
  printf ("Fin du processus %d avec a= %d. \n" , getpid ( ) , a ) ;
  return 0 ;
}
```

61

61

Exemple de la fonction fork()

- Exemple 3
 - Deux exécutions du programme montrent que les processus père et fils sont concurrents :

```
station > cc exemple3.cc exemple3
station > ./exmp3
ici processus pere, le pid 12339.
ici processus fils, le pid 12340.
Fin du Process 12340 avec a = 30.
Fin du Process 12339 avec a = 120.

station > ./fork3
ici processus pere, le pid 15301.
Fin du Process 15301 avec a = 120.
ici processus fils, le pid 15302.
Fin du Process 15302 avec a = 30.
station >
```

62

62

Exemple de la fonction fork()

- Exemple 4: Quelle est la différence entre les programmes exemple4-1 et exemple4-2 suivants ? Combien de fils engendreront-ils chacun ?

```
int main()
{ int i, n=5; int childpid ;
  for (i = 1 ; i<n ; i++)
  { if ( ( childpid=fork () ) <= 0 ) break ;
    printf( " Processus %d avec pere %d , i=%d\n" , getpid() , getppid() , i ) ;
  }
  return 0 ;
}
```

```
int main ( )
{ int  i , n=5; pid_t pid ;
  for ( i = 1 ; i <n ; i ++ )
  { if ( ( pid=fork ( ) ) == -1 ) break ;
    if (pid == 0 )
      printf( " Processus %d avec pere %d , i=%d\n" , getpid ( ) , getppid ( ) , i ) ;
  }
  return 0;
}
```

63

63

Exemple de la fonction fork()

- Exemple 4
 - La sortie de exemple4-1 : À chaque retour de l'appel de fork(), on sort de la boucle si on est dans un processus fils (valeur de retour égale à 0), ou si l'appel a échoué (valeur de retour négative).
 - Le shell, qui est le père du processus créé lors de l'exécution de fork, a un pid de 2206, et le processus lui-même a un pid = 11528.

```
saad@NSAAD:~/Desktop/Chap2$ gcc exemple4_1.c -o exemple4_1
saad@NSAAD:~/Desktop/Chap2$ ./exemple4_1
Processus 11528 avec pere 2206 , i=1
Processus 11528 avec pere 2206 , i=2
Processus 11528 avec pere 2206 , i=3
Processus 11528 avec pere 2206 , i=4
```

64

64

Exemple de la fonction fork()

Exemple 4

```
saad@NSAAD: ~/Desktop/Chap2$ ./exemple4_2
Processus 11560 avec pere 11559 , i=1
Processus 11561 avec pere 11559 , i=2
Processus 11562 avec pere 11559 , i=3
Processus 11563 avec pere 11560 , i=2
Processus 11564 avec pere 11559 , i=4
Processus 11568 avec pere 11563 , i=3
Processus 11570 avec pere 11560 , i=4
Processus 11566 avec pere 2203 , i=3
Processus 11565 avec pere 2203 , i=3
Processus 11567 avec pere 2203 , i=4
Processus 11569 avec pere 2203 , i=4
Processus 11572 avec pere 11568 , i=4
```

- Si l'appel à fork() n'échoue jamais, on sait que le premier processus créera 4 nouveaux processus fils.
- Le premier de ces fils continue d'exécuter la boucle à partir de la valeur courante du compteur **i**. Il créera donc lui-même trois processus.
- Le second fils en créera deux, et ainsi de suite.
- Et tout cela se répète récursivement avec chacun des fils.

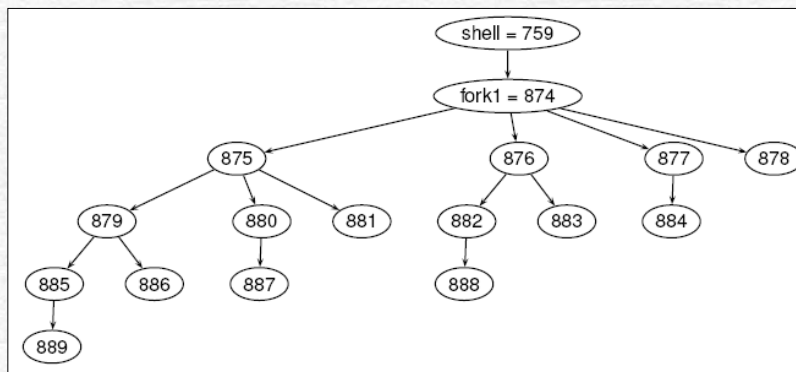
65

65

Exemple de la fonction fork()

Exemple 4

- En supposant que le shell a toujours un **pid** de 759, et que le processus associé à **fork** a un **pid** de 874, on devrait obtenir l'arbre suivante:



66

66

Exemple de la fonction fork()

- Exemple 4
 - Malheureusement, ce ne sera pas le cas, puisque encore une fois des processus pères meurent avant que leur fils ne puissent afficher leurs messages :

```
saad@NSAAD:~/Desktop/Chap2$ ./exemple4_2
Processus 1631 avec pere 1630 , i=1
Processus 1632 avec pere 1630 , i=2
Processus 1633 avec pere 1630 , i=3
Processus 1634 avec pere 1631 , i=2
Processus 1635 avec pere 590 , i=4
Processus 1636 avec pere 1631 , i=3
saad@NSAAD:~/Desktop/Chap2$ Processus 1640 avec pere 1634 , i=3
Processus 1641 avec pere 1632 , i=4
Processus 1637 avec pere 590 , i=4
Processus 1638 avec pere 1632 , i=3
Processus 1643 avec pere 590 , i=4
Processus 1642 avec pere 590 , i=4
Processus 1645 avec pere 1638 , i=4
Processus 1639 avec pere 590 , i=4
Processus 1644 avec pere 590 , i=4
```

67

67

L'appel système exit()

- **void exit (int etat)**
 - La fonction provoque la terminaison du processus avec le code de retour **etat** (0 = bonne fin). Si le père est un shell, il récupère **etat** dans la variable **\$?**
 - A l'exécution de **exit**, tous les fils du processus sont rattachés au processus de pid 1.
 - **exit** réalise la libération des ressources allouées au processus et notamment ferme tous les fichiers ouverts.
 - Si le père est en attente sur **wait**, il est réveillé et reçoit le code de retour du fils.
 - Il faut souligner qu'un processus peut se terminer aussi par un arrêt forcé provoqué par un autre processus avec l'envoi d'un **signal** du type **kill()**.

68

68

L'appel système exit()

- Code de retour de fork:
 - Le code de retour de fork vaut 0 dans le fils
 - Dans le père, c'est l'identifiant du processus (pid) fils
 - En cas d'erreur, ce code de retour vaut -1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv) {
    pid_t respid;
    respid = fork();

    if (respid == -1) {
        /* echec */
        perror("fork ");
        exit(1);
    }

    if (respid == 0) {
        /* fils */
        printf("Je suis le fils\n");
    }
    else {
        /* pere */
        printf("Je suis le pere ");
        printf("(pid fils = %d)\n", respid);
    }
    exit(0);
}
```

69

69

Les appels système wait() et waitpid()

- Appel système: **pid = wait (status)**
 - **pid** est l'identifiant du processus fils et **status** est l'adresse dans l'espace utilisateur d'un entier qui contiendra le status de **exit()** du processus fils.
 - Ces appels système permettent au processus père d'attendre la fin d'un de ses processus fils et de récupérer son **status** de fin.
 - Ainsi, un processus peut synchroniser son exécution avec la fin de son processus fils en exécutant l'appel système **wait()**.

```
#include <sys/wait.h>
int wait (int *status);
int waitpid(int pid, int *status, int options);
```

- **wait()** : permet à un processus père d'attendre jusqu'à ce qu'un processus fils termine. Il retourne l'identifiant du processus fils et son état de terminaison dans **&status**.
- **waitpid()** : permet à un processus père d'attendre jusqu'à ce que le processus fils numéro **pid** termine. Il retourne l'identifiant du processus fils et son état de terminaison dans **&status**.

70

70

Les appels système wait(), waitpid(), exit()

- Le processus appelant est mis en attente jusqu'à ce que l'un de ses fils termine. Quand cela se produit, il revient de la fonction. Si **status** est différent de 0, alors 16 bits d'information sont rangés dans les 16 bits de poids faible de l'entier pointé par **status**.
- Ces informations permettent de savoir comment s'est terminé le processus selon les conventions suivantes :
 - Si le fils est stoppé, les 8 bits de poids fort contiennent le numéro du signal qui a arrêté le processus et les 8 bits de poids faible ont la valeur octale **0177**.
 - Si le fils s'est terminé avec un **exit()**, les 8 bits de poids faible de **status** sont nuls et les 8 bits de poids fort contiennent les 8 bits de poids faible du paramètre utilisé par le processus fils lors de l'appel de **exit()**.
 - Si le fils s'est terminé sur la réception d'un signal, les 8 bits de poids fort de **status** sont nuls et les 7 bits de poids faible contiennent le numéro du signal qui a causé la fin du processus.

71

71

Exemples de wait()

- Exemple 5

```
int main()
{
    pid_t v;
    v=fork();
    if(v==0)
        printf("Je suis le fils avec pid %d\n" , getpid () ) ;
    else if (v > 0) {
        printf("Je suis le pere avec pid %d\n" , getpid () ) ;
        printf("J'attends que mon fils se termine\n" ) ;
        wait(NULL) ;
    }
    else
        printf ("Erreur dans la creation du fils \n" ) ;
    exit(0);
}
```

72

72

Exemples de wait()

Exemple 6

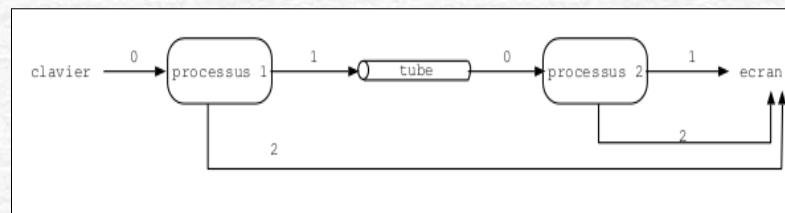
```
int main ()
{ int i, n=5; pid_t pid ;
  for ( i = 1 ; i < n ; i ++ )
  { if ( ( pid=fork ( ) ) == -1 ) break ;
    if ( pid == 0 ) printf ( " Processus %d avec pere %d , i=%d\n" , getpid ( ) , getppid ( ) , i ) ;
  }
  // attendre la fin des fils
  while ( wait ( NULL ) >= 0 ) ;
  return 0 ;
}
```

73

73

La communication inter-processus

- ❑ Le mécanisme `exit+wait` permet
 - au processus fils de retourner une information codée sur 8 bits au processus père
 - au processus père d'attendre la terminaison du fils
- mais il ne permet pas aux processus d'échanger des informations plus complexes
- Solution: **mécanisme de communication par tube** permet
 - d'échanger des informations (éventuellement plus complexes) entre processus
 - de se synchroniser par un mécanisme de lecture/écriture



74

74